

Review Tema 2 - POO

Proiectul „Sistem de Gestiune a Inventarului”, realizat pentru Tema 2 la cursul de Programare Orientată pe Obiecte (POO), este o aplicație practică menită să simuleze activitatea dintr-un depozit de materiale de construcții. Prin intermediul acestui program, utilizatorul poate gestiona stocuri de materiale, baza de furnizori și diverse tipuri de tranzacții, precum achizițiile de noi produse, consumurile pentru lucrări sau returnările. O analiză detaliată a codului sursă demonstrează nu doar respectarea strictă a cerințelor temei, ci și integrarea acestora într-o logică de business funcțională, evitând implementările forțate, adăugate exclusiv pentru punctaj.

Un prim aspect care iese în evidență la deschiderea proiectului este organizarea generală a codului. Conform cerințelor standard ale cursului, codul claselor trebuie separat în fișiere de tip header (.h sau .hpp) și fișiere sursă (.cpp). Spre deosebire de proiectele care se bazează pe un singur fișier main.cpp supradimensionat și greu de urmărit, această aplicație este modularizată corect. Există fișiere dedicate pentru componentele principale: gestiunea inventarului, materiale, furnizori, tranzacții și meniul interactiv. Această structură este extrem de utilă în practică. Atunci când este necesară modificarea logicii unei tranzacții sau rezolvarea unui bug, secțiunea relevantă de cod este ușor de localizat în fișierul corespunzător. Este un prim pas esențial către un cod scalabil și ușor de întreținut de către orice programator.

Partea de moștenire reprezintă nucleul cerințelor pentru Tema 2, iar regula de a avea o clasă de bază proprie cu cel puțin trei clase derivate este îndeplinită printr-o arhitectură bine gândită. Proiectul propune o clasă abstractă de bază denumită `Transaction`, din care derivă patru clase distincte: `PurchaseOrder` (folosită pentru comenzile de aprovizionare de la furnizori), `ConsumptionRecord` (pentru materialele scoase din stoc și folosite pe șantier), `ReturnTransaction` (pentru materialele aduse înapoi în depozit) și `AdjustmentTransaction` (pentru corecțiile manuale de inventar, în cazul unor diferențe la numărătoare). Această abordare ilustrează perfect utilitatea moștenirii într-un scenariu real. Deși la nivel conceptual toate sunt tranzacții, fiecare acționează complet diferit asupra cantităților din depozit în momentul în care este procesată.

O altă cerință importantă solicită utilizarea unui pointer la clasa de bază pentru apelarea funcțiilor virtuale pure. În acest sens, clasa `Inventory` acționează ca un manager central, gestionând o listă (un `std::vector`) de pointeri la clasa abstractă `Transaction`. Pentru polimorfism, se folosește șablonul de proiectare NVI (Non-Virtual Interface). Mai exact, clasa de bază oferă o funcție publică denumită `apply()`, care se ocupă de validările generale (cum ar fi prevenirea procesării repetate a aceleiași

tranzacții pe același document), iar ulterior aceasta apelează funcția privată și virtuală pură `do_apply()`. Fiecare dintre cele patru clase derivate suprascrive exclusiv metoda `do_apply()` cu logica ei matematică. Această structură previne duplicarea codului și garantează că regulile generale ale sistemului nu pot fi ocolite din greșeală. Această separare între interfața publică și implementarea privată este un concept pe care mulți studenți îl ignoră, preferând doar funcțiile virtuale clasice, ceea ce face proiectul cu atât mai deosebit.

În aceeași sferă a polimorfismului, implementarea necesită și o metodă `clone()`, cunoscută în literatura de specialitate drept „constructor virtual”. Aceasta permite copierea în siguranță a unui obiect derivat, chiar și atunci când codul are acces doar la un pointer către clasa de bază. Fără această metodă, copierea unei colecții eterogene de tranzacții ar duce la problema de „object slicing”, pierzându-se attributele specifice claselor derivate (de exemplu, motivul returnării dintr-o tranzacție specifică). Datorită acestui constructor virtual, copierea se realizează complet și corect.

Mai mult, pentru ca managementul resurselor să fie impecabil, clasele principale aplică idiomul Copy-and-Swap. Operatorii de atribuire și constructorii de copiere din clase precum `Material`, `Provider` sau `Inventory` respectă cu strictețe „Regula celor 3” (Rule of 3). Astfel, sunt evitate erorile critice de memorie (precum segmentation faults) care ar apărea la o simplă egalare a două obiecte sau la transmiterea lor prin valoare într-o funcție.

Un avantaj major al implementării este utilizarea pointerilor inteligenți, în special `std::unique_ptr<Transaction>`. Gestiunea manuală a memoriei, folosind cuvintele cheie `new` și `delete`, este adesea o sursă principală de scurgeri de memorie (memory leaks) în proiectele academice. Prin folosirea `unique_ptr`, în momentul în care o tranzacție este ștearsă din istoric sau la închiderea programului, memoria alocată dinamic pe heap se eliberează automat și în siguranță. Această decizie tehnică denotă o înțelegere matură a standardelor moderne de scriere a codului în C++.

Printre conceptele obligatorii se numără și folosirea operatorului `dynamic_cast`. De multe ori, acest operator este introdus artificial de către studenți, doar pentru a acoperi baremul de corectare. În aplicația de față, instrumentul își găsește o utilitate practică reală. Spre exemplu, în cadrul funcțiilor de anulare a unor comenzi, sistemul trebuie să afle exact ce tip de obiect se ascunde sub un pointer generic. Nu se poate anula o achiziție dacă pointerul indică o returnare. Programul efectuează un `dynamic_cast`, verifică la runtime validitatea conversiei (asigurându-se că rezultatul nu este un pointer nul), și abia apoi execută operațiunea de anulare.

Sistemul de tratare a excepțiilor reprezintă o altă componentă bine conturată a temei. În loc să se bazeze pe simple mesaje de eroare afișate prin `std::cout` și pe opriri

bruște ale execuției, codul propune o ierarhie completă de excepții. Există o clasă de bază proprie, `InventoryException`, derivată direct din clasa standard `std::exception`. Din aceasta se ramifică excepții mult mai specializate: `ValidationException` (aruncată la introducerea unor date de contact invalide), `ResourceNotFoundException` (aruncată atunci când utilizatorul caută un furnizor sau un material inexistent) și `InsufficientStockException` (aruncată când se încearcă scoaterea din gestiune a unei cantități mai mari decât stocul existent). Faptul că aceste excepții sunt aruncate direct din constructori garantează că programul nu va lucra niciodată cu obiecte aflate într-o stare invalidă. La nivelul interfeței (în meniul din consolă), excepțiile sunt capturate prin blocuri `try/catch`, care afișează erorile într-un mod prietenos și permit utilizatorului să reintroducă datele fără ca programul să cedeze.

Integrarea elementelor statice este la fel de bine justificată și logică. Clasa de meniu folosește șablonul Singleton, având un constructor privat și permițând accesul doar printr-o funcție statică `get_instance()`. De asemenea, programul utilizează constante statice pentru date configurabile, un exemplu fiind parola `DEV_PASSWORD`, care deblochează funcționalități ascunse de tip "Developer Mode" – un detaliu de finețe adăugat peste cerințele de bază. La capitolul rigurozitate, se observă și o respectare constantă a conceptului de const-correctness. Metodele de afișare și parametrii transmiși prin referință doar pentru citire sunt marcați cu `const`, protejând datele împotriva oricăror suprascrieri accidentale.

Nu în ultimul rând, proiectul face un uz intensiv de Standard Template Library (STL). Colecțiile de date folosesc `std::vector`, manipularea textelor se face exclusiv prin `std::string`, iar codul face tranziția către programarea modernă din standardul C++20, integrând algoritmi din librăria `<ranges>`. Folosirea unor filtre cu `std::ranges::find_if` alături de expresii lambda face codul mai concis și mai expresiv decât utilizarea unor bucle `for` clasice. Mai mult, pentru validarea structurii adreselor de email și a numerelor de telefon, a fost integrată librăria `std::regex`, un element care sporește semnificativ gradul de profesionalism al aplicației.

În concluzie, proiectul analizat ilustrează o asimilare profundă a conceptelor de Programare Orientată pe Obiecte. Deși respectarea unei liste detaliate de cerințe tehnice (precum metode virtuale pure, clona, NVI, excepții personalizate sau Copy-and-Swap) poate genera adesea un cod dezordonat sau artificial, în cazul acestui sistem de gestiune a inventarului, absolut toate aceste concepte se îmbină organic. Moștenirea facilitează logica diferitelor tranzacții, polimorfismul permite procesarea lor unitară, pointerii inteligenți gestionează memoria în fundal, iar meniul transformă totul într-un program interactiv stabil. Este un exemplu de temă realizată la un standard foarte ridicat, care îndeplinește criteriile de evaluare într-un mod corect și logic.